

ITCS 6169/8169 Assignment 1 — The CNN Challenge

Anusha Reddy · Code, configs, logs & final checkpoint: <https://github.com/<your-username>/<your-repo>>

1. Final result

Test accuracy: 96.75 % (387/400) with a single fine-tuned **ConvNeXt-Tiny** CNN. **Validation strategy:** stratified 80/20 split of the 2,400 training images (120/30 per class, seed 0), identical for every experiment; model selection = epoch with the best validation accuracy. The test set was evaluated *once*, after the final model had been chosen on validation and that choice committed to git. **Validation accuracy of the selected model: 96.67 % (464/480);** the test number agrees to within one image. Reproduce: `python train.py --config configs/best.yaml; python evaluate.py --checkpoint checkpoints/final_convnext_tiny_fp16.pt`.

2. The recipe, and which choices mattered

Architecture/initialisation: ConvNeXt-Tiny (28 M params) initialised from torchvision IMAGENET1K_V1 weights; the 1000-way head is replaced by a 16-way linear layer and *all* layers are fine-tuned. The 256 px greyscale images are replicated to 3 channels and normalised with ImageNet statistics; input 224×224 .

Optimisation: AdamW, lr 10^{-4} , weight decay 0.05 (not on norms/biases), 1 warm-up epoch + cosine decay, batch 32, 30 epochs (40 min on an Apple M4). **Regularisation:** label smoothing 0.1; *light* augmentation only (reflect-pad random crop + horizontal flip). No test-time augmentation (it lowered val accuracy).

What mattered, in order: (1) *ImageNet initialisation* (+5.8 % for the same ResNet-18; a frozen-backbone linear probe already beats every from-scratch model); (2) *task-appropriate augmentation*: light > strong by +1.0 % (2 seeds); (3) *label smoothing* (+1.5 % mean over 3 seeds, every seed improved); (4) the *learning rate* for ConvNeXt ($3e-4 \rightarrow 1e-4$: +0.8 %, far more stable curve); (5) *resolution*: 288 px gave +1.4 % for ResNet-18 and +1.1 % for EfficientNet-B0. The *CNN family* mattered least: five ImageNet CNNs from 0.4 to 9 GFLOPs land within ~ 1 % of each other (Fig. 1). **Efficiency pick:** EfficientNet-B0 @288 px reaches 96.0 % val at 1/7 of the FLOPs.

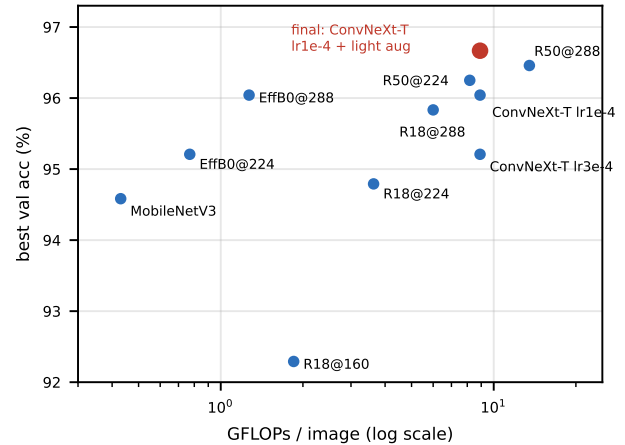


Fig. 1: val accuracy vs. compute (ImageNet-pretrained CNNs; R = ResNet).

3. Experimental journey

Experiment (one change at a time)	Val acc.	Observation
Baseline: starter TNet, grey 64 px	44.0 %	99 % train acc.: overfits, and features too weak
Own 8-layer CNN (BN, GAP), 128 px, AdamW+cosine	87.7 %	depth/GAP head $\gg$ parameter count (1.2M)
+ strong / light augmentation	87.3 / 90.2 %	strong aug <i>hurt</i> (Sec. 4); light aug helps
ResNet-18 from scratch, 224 px	86.7 %	10 $\times$ the capacity, no gain without pretraining
+ ImageNet init (all layers fine-tuned)	92.9 % [†]	pretraining is the largest single gain
+ label smoothing 0.1	94.4 % [†]	consistent over 3 seeds (std 0.6 %)
+ MixUp/CutMix	92.9 %	hurt: over-regularises a 30-epoch run
+ light instead of strong aug	95.4 % [†]	the scratch-model finding transfers
Family (224 px): MobileNetV3, EffNet-B0, ResNet-50, ConvNeXt-T	94.6–95.7 %	94.6 / 94.9 [†] / 95.7 [†] / 95.2 %: within ~ 1 %
ConvNeXt-T, lr $3e-4 \rightarrow 1e-4$	96.5 % [†]	lr was limiting ConvNeXt, not capacity
Final: ConvNeXt-T, lr $1e-4$, light aug	96.7 %	test 96.75 %

[†]mean over 2–3 training seeds (same split). One val image = 0.21 %; single-seed differences <1 % were treated as noise and re-run.

4. Failure analysis: strong augmentation made things worse

What I tried: the standard ImageNet-style pipeline (RandomResizedCrop with scale 0.35–1, brightness/contrast jitter, $\pm 10^\circ$ rotation, flip) on the from-scratch CNN, which was at 100 % train accuracy. **Why I expected it to work:** augmentation is the usual remedy for overfitting on 1,920 images. **What happened:** validation accuracy *dropped* (87.7 $\rightarrow$ 87.3 %) and train accuracy fell to 90 %, with the best epoch being the last. I tested three explanations, one change each: (i) *under-training*: 150 instead of 60 epochs reached 97 % train accuracy but only 88.1 % val, so convergence was not the problem; (ii) *aggressive crops*: restricting the crop scale to 0.8–1 recovered part of the loss (88.8 %); (iii) *only label-preserving geometry* (pad-crop + flip): 90.2 %, the best scratch result. **What I learned:** scene classes are defined by *global layout* (horizon position, sky/ground ratio, vertical structure). A 35 %-area crop of a *Coast* image is plausibly *OpenCountry*, and rotation adds black corners never seen at test time, so the augmented training distribution drifts away from the test distribution instead of covering it. Augmentation must be label-preserving *for the task*. The lesson transferred: light augmentation also improved the pretrained ResNet-18 (+1.0 %, 2 seeds) and is part of the final

model. Two smaller surprises: MixUp/CutMix hurt (−1.9%), and ResNet-50’s first run (96.3%) was partly luck: its best-vs-last-epoch gap was 1.7% and a second seed gave 95.2%.

Error analysis. The hardest classes are consistent across models: *InsideCity* and *Bedroom* (confused with *LivingRoom*) on validation. 9 of the 13 test errors involve *OpenCountry* (e.g. grassy hillsides labelled *Mountain*); several are arguably ambiguous labels. Pretraining helped most on *OpenCountry* (+17%) and *Industrial* (+13%), and not at all on *InsideCity*.

5. AI + Human

I used **Claude Code** as a pair programmer (details in `AI_USAGE.md`). **Where it helped:** it turned the starter notebook into a config-driven, logged pipeline in minutes and built a throughput benchmark that located the real bottleneck. **Where judgement was needed:** its suggested `channels_last` speed-up (good advice on CUDA) made training 2–3× *slower* on the Apple GPU; it was measured and removed. More importantly, a first-draft analysis written before the numbers were checked mis-stated which classes benefit from pretraining. Every claim in this report was re-derived from the logged results, and every decision-driving comparison was repeated with extra seeds before being believed.